ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ

Мэдээлэл, холбооны технологийн сургууль

ЛАБОРАТОРИЙН АЖИЛ №3

Үйлдлийн систем

2026–2027 оны хичээлийн жилийн намрын улирал

Лабораторийн ажлын сэдэв: Бүлэг 3. Процесс (*Processes*)

Practice Exercises 3.1–3.17

Лабораторийн ажлыг гүйцэтгэсэн оюутан:

_____ /_____/

Улаанбаатар хот

2026 он

Агуулга

1 Лабораторийн зорилго **3**

2 Practice Exercises 3.1–3.17 **4**

3.1 4
3.2 5
3.3 7
3.4 8
3.5 9
3.6 9
3.7 11
3.8 12
3.9 13
3.10 14
3.11 16
3.12 17
3.13 19
3.14 20
3.15 21
3.16 23
3.17 24

3 Дүгнэлт **27**

Ашигласан материал **27**

1 Лабораторийн зорилго

Энэхүү лабораторийн ажлаар A. Silberschatz, P. B. Galvin, G. Gagne нарын *Operating System Concepts* (10-р хэвлэл, Wiley, 2018) сурах бичгийн 3-р бүлэг «Процесс»-ийн **Practice Exercises 3.1–3.17** даалгаврыг гүйцэтгэв. Үүнээс 3.1–3.7 нь бүлгийн төгсгөлд байх «Practice Exercises» хэсэгт (х. 154–155), 3.8–3.17 нь «Chapter 3 Exercises» хэсэгт (х. EX-4–EX-7) байрладаг. Асуулт бүрийг монгол хэлээр орчуулан бичиж, хариултыг номын холбогдох хэсгүүдэд (§3.2.3, §3.3, §3.6, §3.7.4, §3.8.2) тулгуурлан тайлбарлав.

Програмын кодтой холбоотой асуултуудын (3.1, 3.2, 3.11, 3.12, 3.13, 3.16) хариултыг Linux орчинд кодыг хөрвүүлж ажиллуулан баталгаажуулав (Ubuntu 24.04, `gcc` 13.3.0). 3.9, 3.10, 3.14-ийн жишээг тухайн орчны бодит `ps`, `kill`, `mkfifo` командын гаралтаар үзүүлэв.

2 Practice Exercises 3.1–3.17

3.1

Асуулт 3.1

Зураг 3.30-д үзүүлсэн програмыг ашиглан `LINE A` дээр ямар утга хэвлэгдэхийг тайлбарла.

```c
#include <sys/types.h>
#include <sys/wait.h>
#include <stdio.h>
#include <unistd.h>

int value = 5;

int main()
{
pid_t pid;

    pid = fork();

    if (pid == 0) { /* child process */
        value += 15;
        return 0;
    }
    else if (pid > 0) { /* parent process */
        wait(NULL);
        printf("PARENT: value = %d",value); /* LINE A */
        return 0;
    }
}
```

Зураг 3.30. LINE A дээр ямар утга гарах вэ? (<sys/wait.h>-г хөрвүүлэхийн тулд нэмсэн)

Хариулт. `LINE A` дээр **`PARENT: value = 5`** гэж хэвлэгдэнэ.

Тайлбар:

1. `fork()` дуудагдахад хүүхэд процесс нь эцэг процессын **хаягийн орон зайн хуулбар** (код, глобал өгөгдөл, стек, heap) болж үүснэ. Иймд глобал хувьсагч `value` нь эцэг болон хүүхэд процесст хоорондоо хамааралгүй хоёр тусдаа хуулбараар оршино.
2. Хүүхэд процесс (`pid == 0`) `value += 15`-г **өөрийн** хуулбар дээр гүйцэтгэж утгыг $5 \to 20$ болгоод дуусна. Энэ өөрчлөлт эцгийн хуулбарт огт нөлөөлөхгүй.
3. Эцэг процесс (`pid > 0`) `wait(NULL)`-аар хүүхдийг дуустал хүлээгээд өөрийн `value`-г хэвлэнэ. Эцгийн хуулбар өөрчлөгдөөгүй тул утга нь анхны **5** хэвээр байна.

Хэрэв эдгээр нь процесс биш, нэг хаягийн орон зайг хуваалцдаг thread байсан бол хувьсагч нэг л хуулбартай тул үр дүн 20 болох байсан. (Орчин үеийн UNIX/Linux *copy-on-write* техник ашигладаг тул хуулбар нь физикээр хүүхэд өгөгдөл рүү бичих үед л үүсдэг ч логик үр дүн адилхан.)

Баталгаажуулалт. Хүүхэд, эцэг хоёул `value`-г хэвлэдэг болгож өөрчилсөн програмыг ажиллуулахад:

```
CHILD:  value = 20 (address 0x55e2bb4f3010)
PARENT: value = 5 (address 0x55e2bb4f3010)
```

Хоёр процесст `value`-ийн виртуал хаяг ижил боловч утга өөр байгаа нь хаягийн орон зай тус тусдаа гэдгийг харуулж байна. Анхны програм `PARENT: value = 5` гэж хэвлэв.

3.2

Асуулт 3.2

Анхны эцэг процессыг оролцуулан тооцвол Зураг 3.31-д үзүүлсэн програм хэдэн процесс үүсгэх вэ?

```c
#include <stdio.h>
#include <unistd.h>

int main()
{
    /* fork a child process */
    fork();

    /* fork another child process */
    fork();

    /* and fork another */
    fork();

    return 0;
}
```

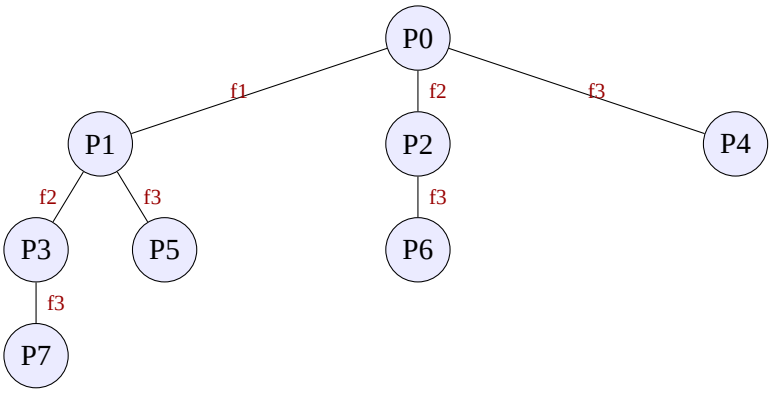
Зураг 3.31. Хэдэн процесс үүсэх вэ?

Хариулт. Нийт **8 процесс** үүснэ (анхны эцэг процесс + 7 шинэ хүүхэд процесс).

Тайлбар: `fork()` дуудагдах үед тухайн мөчид байгаа **бүх** процесс дараагийн мөр рүү шилжиж, тус бүр нь дараагийн `fork()`-ийг дахин дууддаг. Иймд `fork()` бүр процессын тоог 2 дахин нэмэгдүүлнэ:

Дуудалт	`fork()`-ийн өмнө	Шинээр үүссэн	`fork()`-ийн дараа
1-р `fork()`	1	1	2
2-р `fork()`	2	2	4
3-р `fork()`	4	4	8

Ерөнхийдөө болзолгүй дараалсан n ширхэг `fork()` нийт 2^n процесс үүсгэдэг тул энд $2^3 = 8$; шинээр үүссэн хүүхэд $1 + 2 + 4 = 7$.



Зураг 1. Гурван `fork()`-оор үүсэх процессын мод (P0 – анхны эцэг; `fN` – хүүхдийг үүсгэсэн N-р `fork()` дуудалт).

Баталгаажуулалт. Процесс бүр өөрийн `pid`, `ppid`-г нэг мөрөөр хэвлэдэг болгож ажиллуулав (эцэг нь `wait()`-ээр хүүхдүүдээ хүлээдэг):

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    fork();   /* fork #1 */
    fork();   /* fork #2 */
    fork();   /* fork #3 */

    printf("pid=%d ppid=%d\n", getpid(), getppid());
    while (wait(NULL) > 0) ;    /* wait for all children */
    return 0;
}
```

Гаралт (`pid`-ээр эрэмбэлсэн; `110` нь анхны эцэг, `83` нь түүнийг ажиллуулсан shell):

```
pid=110 ppid=83
pid=111 ppid=110
pid=112 ppid=110
pid=113 ppid=111
pid=114 ppid=111
pid=115 ppid=112
pid=116 ppid=113
pid=117 ppid=110
```

Нийт **8 мөр** гарсан нь 8 процесс үүссэнийг батална. Эцэг `110`-ийн гурван хүүхэд (`111, 112, 117`), `111`-ийн хоёр хүүхэд (`113, 114`), `112`, `113`-ийн тус бүр нэг хүүхэд (`115`, `116`) байгаа нь дээрх модтой яг таарч байна. Зураг 1-ийн зангилаанууд дараах `pid`-тэй тохирно: P0 = 110, P1 = 111, P2 = 112, P3 = 113, P4 = 117, P5 = 114, P6 = 115, P7 = 116.

3.3

Асуулт 3.3

Apple-ийн гар утасны iOS үйлдлийн системийн анхны хувилбарууд зэрэгцээ боловсруулалтыг (*concurrent processing*) дэмждэггүй байсан. Зэрэгцээ боловсруулалт үйлдлийн системд нэмж оруулдаг гурван гол хүндрэлийг хэлэлц.

Хариулт. Олон процессыг нэгэн зэрэг ажиллуулахад үйлдлийн систем дараах гол асуудлуудыг шийдэх шаардлагатай болдог:

1. **CPU-ийн хуваарилалт (*scheduling*) ба контекст сэлгэлт.** Олон процесс цөөн тооны CPU цөмийг хуваалцдаг тул үйлдлийн систем аль процесст, хэдий хугацаанд CPU олгохыг шийддэг хуваарилагч (*scheduler*), мөн бэлэн байгаа, хүлээж байгаа процессуудын дарааллыг (*ready queue*, *wait queue*) зохион байгуулах ёстой. Процесс бүрийн төлөвийг PCB-д хадгалж, процесс солигдох бүрд контекстийг хадгалах, сэргээх шаардлагатай ба энэ хугацаанд ашигтай ажил хийгддэггүй тул нэмэгдэл зардал (*overhead*) гарна.
2. **Санах ойн удирдлага ба хамгаалалт.** Олон процессын код, өгөгдлийг нэгэн зэрэг санах ойд байлгаж, процесс бүрт тусдаа хаягийн орон зай олгох ёстой. Нэг процесс нөгөө процессын эсвэл үйлдлийн системийн санах ойд халдаж өгөгдлийг нь эвдэхээс хамгаалах механизм (хаягийн орон зайн тусгаарлалт, эрхийн хяналт) хэрэгтэй. Санах ой дүүрэх үед аль процессыг санах ойноос гаргах эсвэл зогсоохыг шийдэх шаардлагатай ба гар утсанд санах ой, батарей хязгаарлагдмал тул энэ асуудал онцгой хүнд байдаг.
3. **Синхрончлол ба процесс хоорондын харилцаа (IPC).** Хамтран ажиллаж буй процессууд нийтлэг өгөгдөл, нөөц (файл, төхөөрөмж) ашиглахад гүйцэтгэлийн дарааллаас хамааарч үр дүн буруу гарах **өрсөлдөөний нөхцөл** (*race condition*), **түгжрэл** (*deadlock*), нөөц хүрэлцэхгүй удаан хүлээх (*starvation*) асуудал үүснэ. Иймд

синхрончлолын хэрэгсэл (mutex, semaphore г.м.) болон мэдээлэл солилцох механизм (*shared memory*, *message passing*) шаардлагатай болдог.

Эдгээр хүндрэл, мөн гар утасны хязгаарлагдмал нөөцийн улмаас iOS-ийн эхэн үеийн хувилбарууд хэрэглэгчийн аппликейшнд multitasking-г зөвшөөрөлгүй, зөвхөн нэг аппликейшн foreground-д ажиллаж бусад нь түр зогсдог байв. iOS 4-өөс эхлэн хязгаарлагдмал хэлбэрийн multitasking нэвтэрсэн.

3.4

Асуулт 3.4

Зарим компьютерийн систем олон регистрийн багцтай (*multiple register sets*) байдаг. Шинэ контекст аль хэдийн регистрийн багцуудын нэгэнд ачаалагдсан байх үед контекст сэлгэлт хийгдэхэд юу болохыг тайлбарла. Шинэ контекст регистрийн багцад бус санах ойд байгаа бөгөөд бүх регистрийн багц ашиглагдаж байвал юу болох вэ?

Хариулт. (a) Шинэ контекст аль хэдийн регистрийн багцын нэгэнд байгаа үед. Контекст сэлгэлт нь зөвхөн «одоогийн регистрийн багц»-ыг заасан заагчийг (*pointer*) шинэ процессын багц руу шилжүүлэхээс өөр ажил шаардахгүй. Регистрийн утгуудыг санах ой руу хуулах, санах ойгоос буцааж ачаалах шаардлагагүй тул сэлгэлт **маш хурдан**, хамгийн бага нэмэгдэл зардалтай явагдана.

(b) Шинэ контекст санах ойд (PCB-д) байгаа бөгөөд бүх регистрийн багц завгүй үед. Үйлдлийн систем регистрийн багцуудаас нэгийг «золиос» болгон сонгоно (жишээ нь хамгийн удаан ашиглагдаагүй багцыг, LRU зэрэг бодлогоор). Дараа нь:

1. сонгосон багцын агуулгыг (түүнийг эзэмшиж байсан процессын) PCB-д санах ойд хадгална;
2. шинэ процессын хадгалагдсан контекстийг санах ойгоос уг багцад ачаална;
3. «одоогийн регистрийн багц» заагчийг шинэчилнэ.

Өөрөөр хэлбэл энэ тохиолдолд ердийн (нэг регистрийн багцтай системийн) контекст сэлгэлттэй адилаар регистрийн өгөгдлийг санах ойд хадгалж, санах ойгоос сэргээх ажил хийгдэх тул олон регистрийн багцын хурдны давуу тал алдагдана. Номд бичсэнээр, идэвхтэй процессын тоо регистрийн багцын тооноос олон бол систем регистрийн өгөгдлийг санах ой руу хуулж, буцааж ачаалдаг ердийн аргад шилждэг.

3.5

Асуулт 3.5

Процесс `fork()` үйлдлээр шинэ процесс үүсгэхэд дараах төлөвүүдийн аль нь эцэг болон хүүхэд процессуудын хооронд **хуваалцагдах** вэ?

a. Стек (*Stack*)

b. Heap

c. Хуваалцсан санах ойн сегментүүд (*Shared memory segments*)

Хариулт. Зөвхөн **c. Хуваалцсан санах ойн сегментүүд** хуваалцагдана.

- **a. Стек – хуваалцагдахгүй.** Хүүхэд процесс эцгийн стекийн хуулбарыг (локал хувьсагч, функцийн буцах хаяг г.м.) өөртөө авна.
- **b. Heap – хуваалцагдахгүй.** Динамикаар хуваарилагдсан санах ойн (`malloc()`) хуулбарыг хүүхэд авах тул нэг процессын хийсэн өөрчлөлт нөгөө процесст харагдахгүй.
- **c. Хуваалцсан санах ойн сегмент – хуваалцагдана.** `fork()` үед хуваалцсан сегмент хуулагдахгүй, хоёр процессын хаягийн орон зайд ижил физик санах ойг заасан хэвээр үлдэнэ. Иймд нэг процессын бичсэн өгөгдлийг нөгөө нь шууд харна. Үүнд POSIX shared memory (`shm_open()`, `mmap()` ба `MAP_SHARED` тугийн) §3.7.1-д тайлбарласан процесс хоорондын харилцаа тулгуурладаг.

Учир нь: `fork()` нь эцгийн хаягийн орон зайн *хувийн* хэсгүүдийн (стек, heap, өгөгдөл, код) хуулбарыг хүүхдэд өгдөг бол `MAP_SHARED`-аар холбогдсон сегмент нь зориудаар хуваалцагдахаар тохируулагдсан байдаг.

3.6

Асуулт 3.6

RPC механизмтай холбоотой «яг нэг удаа» (*exactly once*) семантикийг авч үз. Клиент рүү буцаасан ACK мессеж сүлжээний доголдлоос болж алга болсон ч уг семантикийг хэрэгжүүлэх алгоритм зөв ажиллах уу? Мессежүүдийн дарааллыг тайлбарлаж, «яг нэг удаа» хэвээр хадгалагдах эсэхийг хэлэлц.

Хариулт. **Тийм**, ACK алдагдсан ч алгоритм зөв ажиллаж, «яг нэг удаа» семантик **хадгалагдана**.

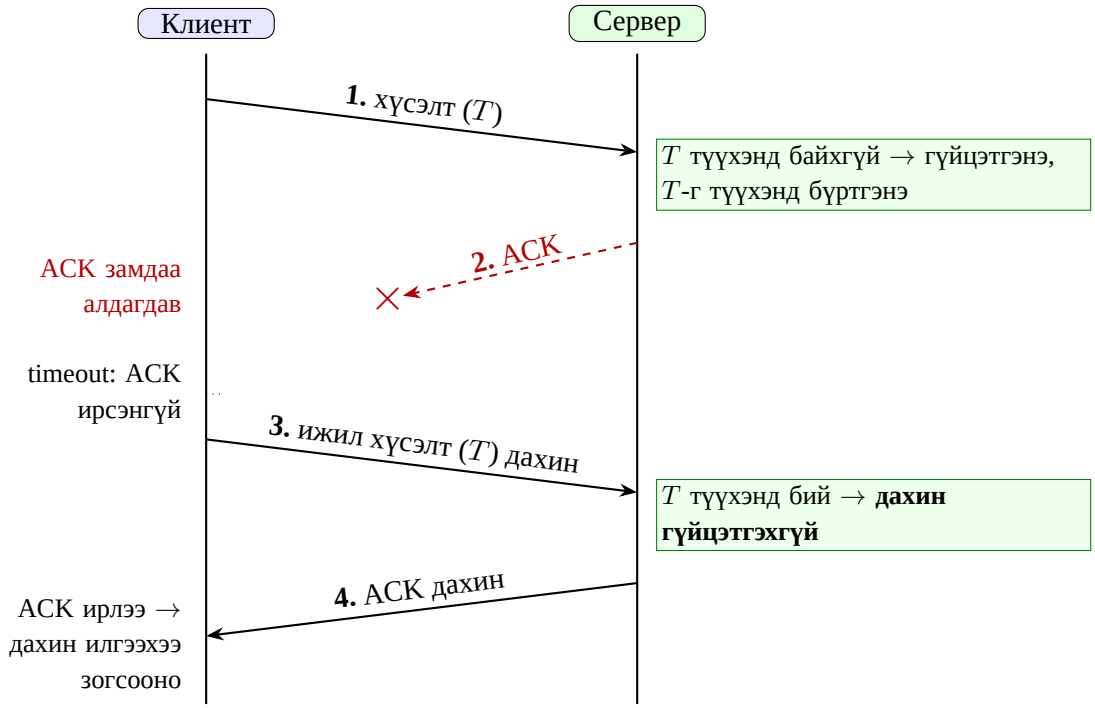
Ном (§3.8.2)-ын дагуу «яг нэг удаа» нь хоёр шаардлагын нийлбэр юм:

- **Хамгийн ихдээ нэг удаа** (*at most once*) – RPC мессеж бүрт цагийн тэмдэг (*timestamp*) хавсаргана. Сервер боловсруулсан мессежүүдийн цагийн тэмдгийн түүхийг (*history*) хадгалж, түүхэнд байгаа цагийн тэмдэгтэй мессежийг давхардсан гэж үзэн дахин гүйцэтгэхгүй.

- **Хамгийн багадаа нэг удаа** (*at least once*) – сервер хүсэлтийг хүлээн авч гүйцэтгэснээ клиентэд ACK мессежээр мэдэгдэнэ. Клиент ACK хүлээн авах хүртэл RPC-г үе үе дахин илгээнэ.

Мессежүүдийн дараалал (ACK алдагдсан тохиолдол):

1. Клиент RPC хүсэлтийг цагийн тэмдэг T-тэй сервер рүү илгээнэ.
2. Сервер хүсэлтийг хүлээн авч, T түүхэнд байхгүйг шалгаад процедурыг **нэг удаа** гүйцэтгэн T-г түүхэнд бүртгэж, ACK илгээнэ.
3. ACK сүлжээнд алга болж, клиентэд хүрэхгүй.
4. Тогтоосон хугацаанд (*timeout*) ACK ирээгүй тул клиент яг ижил хүсэлтийг (ижил T-тэй) **дахин илгээнэ**.
5. Сервер давхардсан хүсэлтийг хүлээн авч, T түүхэнд байгааг олж мэдээд процедурыг **дахин гүйцэтгэхгүй**, харин ACK-г **дахин илгээнэ**.
6. Клиент ACK-г хүлээн авч, дахин илгээхээ зогсооно.



Зураг 2. ACK алдагдсан үеийн «яг нэг удаа» RPC-ийн мессежийн дараалал.

Дүгнэлт. Процедур яг нэг удаа гүйцэтгэгдсэн: түүхийн шалгалт давхар гүйцэтгэлээс сэргийлсэн (*at most once*), клиентийн дахин илгээлт хүсэлт серверт хүрэхийг баталгаажуулсан (*at least once*). Давхардсан хүсэлтэд сервер зөвхөн ACK-г дахин илгээх ёстой; хүсэлтийг чимээгүй орхивол клиент хэзээ ч ACK авахгүй, үүрд дахин илгээх болно. Мөн хүсэлт өөрөө (ACK биш) алдагдсан бол клиентийн timeout-ийн дараах дахин илгээлтээр сэргээгдэнэ. Түүх нь клиент дахин илгээж болох хугацаанд цагийн тэмдгүүдийг хадгалах хэмжээтэй байх ёстой.

3.7

Асуулт 3.7

Тархсан систем серверийн доголдолд (*server failure*) өртөмтгий гэж үзье. RPC-ийн «яг нэг удаа» гүйцэтгэх семантикийг баталгаажуулахад ямар механизмууд шаардлагатай вэ?

Хариулт. Асуудал. Хэвийн үед сервер «яг нэг удаа»-ийг хүлээн авсан хүсэлтүүдийн түүхийг (цагийн тэмдэг) **санах ойд** хадгалж хангадаг. Сервер доголдож дахин ачаалагдахад санах ойн түүх устах тул сервер тухайн хүсэлтийг гүйцэтгэсэн эсэхээ мэдэхгүй болно. Улмаар клиентийн дахин илгээсэн хүсэлтийг **давхар гүйцэтгэх** (at most once зөрчигдөх) эсвэл гүйцэтгээгүй орхих (at least once зөрчигдөх) эрсдэлтэй.

Шаардлагатай механизмууд.

1. **Тогтвортой хадгалалт дахь лог (*stable storage, log*).** Сервер хүлээн авсан RPC бүрийн тухай мэдээллийг – хүсэлтийн танигч/цагийн тэмдэг, гүйцэтгэгдсэн эсэх төлөв, гүйцэтгэлийн үр дүн – диск зэрэг **доголдолд тэсвэртэй хадгалалтад** бичиж байх ёстой. Ингэснээр сервер унаж дахин асахад түүхээ логоос сэргээнэ. Ном дахь санаа нь: серверт «ямар RPC хүлээн авсан, амжилттай гүйцэтгэсэн эсэх, үр дүн нь юу байсан» гэсэн мэдээллийг тогтвортой хадгалалтад хөтлөх явдал юм.
2. **Гүйцэтгэл ба бүртгэлийн атомар байдал.** Үйлдлийг гүйцэтгэх, түүний үр дүнг логт бичих хоёр нь хуваагдашгүй (transaction маягийн) үйлдэл байх ёстой: эхлээд «эхэлсэн» тэмдэглэгээг бичиж (*write-ahead log*), дуусгаад «дууссан + үр дүн» гэж бичнэ. Ингэвэл үйлдлийн дундуур унасан ч дутуу үйлдлийг буцаах (*rollback*) эсвэл (идемпотент бол) дахин гүйцэтгэх боломжтой.
3. **Сэргэлтийн (*recovery*) алхам.** Сервер дахин асахдаа логийг уншиж: дууссан хүсэлт дахин ирвэл **дахин гүйцэтгэлгүй** хадгалсан үр дүн/ACK-г буцаана; эхэлсэн боловч дуусаагүй хүсэлтийг логийн дагуу зөв дуусгах эсвэл буцаана.
4. **Клиентийн дахин илгээлт ба ACK.** Клиент timeout-оор ACK ирэх хүртэл хүсэлтээ дахин илгээнэ (сервер сэргэсний дараа хүсэлт хүрнэ). Сервер ачаалагдах бүрд «үе»-ийн дугаар (*epoch*) өөрчилдөг бол клиент серверийг дахин асаснаа мэдэж болно.
5. **(Нэмэлт) Репликаци.** Түүх, логийг нөөц серверт хуулбарлан хадгалбал үндсэн сервер унахад нөөц сервер залгамжилж ажиллана.

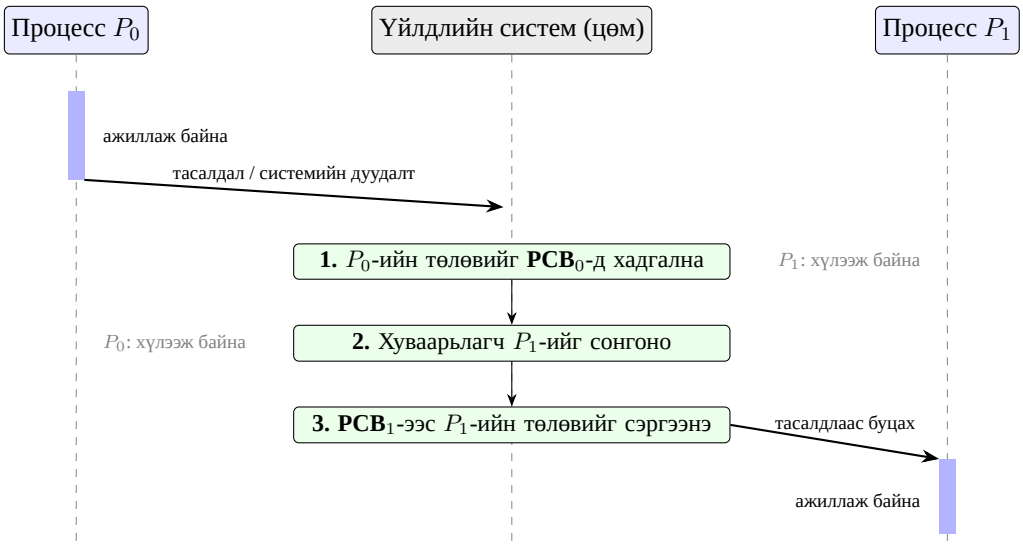
Товчоор, «яг нэг удаа»-г серверийн доголдолд ч хадгалахын тулд **хүсэлтийн түүх, гүйцэтгэлийн төлөв, үр дүнг тогтвортой хадгалалтад (лог) хадгалж**, сэргэсний дараа түүнд тулгуурлан давхар гүйцэтгэлээс сэргийлэх шаардлагатай.

3.8

Асуулт 3.8

Процессуудын хооронд контекст сэлгэхдээ цөм (*kernel*) ямар үйлдлүүдийг гүйцэтгэдгийг тайлбарла.

Хариулт. Контекст сэлгэлт (*context switch*) нь CPU-г ажиллаж буй процесс P_0-оос өөр процесс P_1 руу шилжүүлэх үйлдэл бөгөөд цөм дараах алхмуудыг дарааллаар гүйцэтгэнэ (§3.2.3, Зураг 3.6).



Зураг 3. Контекст сэлгэлт: P_0-оос P_1 руу шилжих үеийн цөмийн үйлдлүүд (номын Зураг 3.6-тай төстэй).

Дээрх зурагт гол үе шатыг товч үзүүлсэн бөгөөд тус бүрийг доор дэлгэрэнгүй тайлбарлав.

1. **Цөмд шилжих.** Тасалдал (цагийн таймер, I/O дуусах) эсвэл системийн дуудалт (жишээ нь процесс өөрөө блоклогдох) гарахад CPU хэрэглэгчийн горимоос цөмийн горимд шилжиж, цөмийн тасалдал боловсруулах горим ажиллана.
2. **Хуучин контекстийг хадгалах (*state save*).** Цөм ажиллаж байсан P_0-ийн контекстийг түүний **PCB**-д (*Process Control Block*) хадгална: CPU-ийн регистрүүд (программ тоологч, стекийн заагч, ерөнхий зориулалтын регистрүүд, төлөвийн регистр), процессын төлөв, санах ойн удирдлагын мэдээлэл (хуудасны хүснэгтийн суурь г.м.), CPU ашигласан хугацааны бүртгэл.
3. **Дараалалд оруулах.** P_0-ийн төлөвийг өөрчилж (*running* $\rightarrow$ *ready* эсвэл *waiting*) бэлэн дараалалд (*ready queue*) эсвэл тохирох хүлээлгийн дараалалд оруулна.
4. **Дараагийн процессыг сонгох.** CPU хуваарьлагч (*scheduler*) бэлэн дарааллаас хуваарилалтын алгоритмын дагуу P_1-ийг сонгоно.
5. **Шинэ контекстийг сэргээх (*state restore*).** P_1-ийн PCB-д өмнө нь хадгалагдсан регистрүүд, программ тоологч, стекийн заагчийг CPU-д ачаалж, P_1-ийн хаягийн орон зайг идэвхжүүлнэ (хуудасны хүснэгтийн суурь регистрийг солих, шаардлагатай бол TLB-г цэвэрлэх). P_1-ийн төлөв *running* болно.

6. **Хэрэглэгчийн горимд буцах.** Цөм тасалдлаас буцаж (*return from interrupt*) P_1 өөрийн өмнө зогссон командаас үргэлжлүүлэн ажиллана.

Сэлгэлтийн үед систем ямар ч ашигтай ажил гүйцэтгэдэггүй тул энэ хугацаа цэвэр **нэмэгдэл зардал** (*overhead*) юм. Сэлгэлтийн хурд нь регистрийн тоо, санах ойн хурд, бүх регистрийг нэг командаар хадгалах/ачаалах тусгай дэмжлэг зэрэг техник хангамжаас хамаарна (3.4-ийг үз).

3.9

Асуулт 3.9

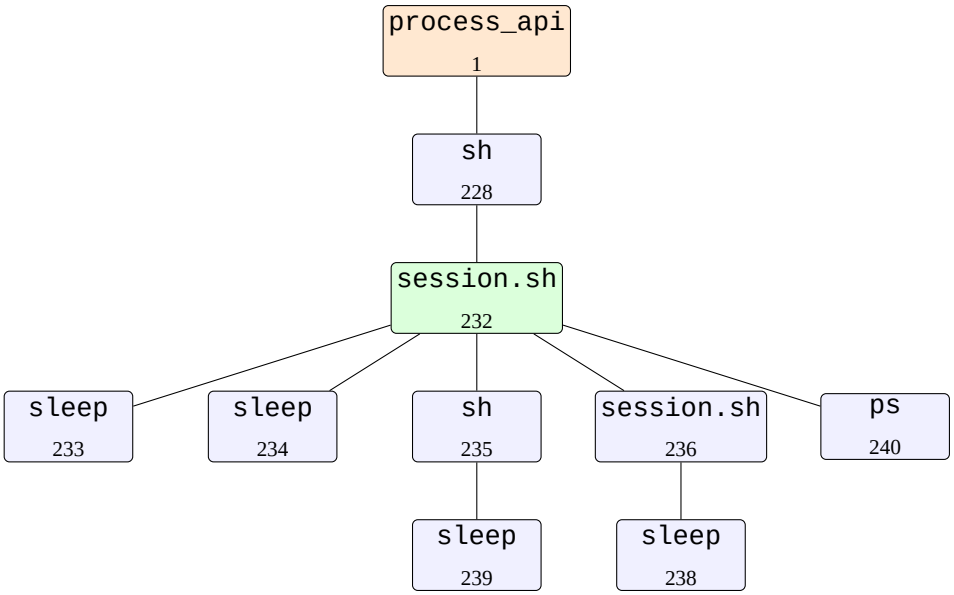
Зураг 3.7-той төстэй процессын мод байгуул. UNIX эсвэл Linux системийн процессын мэдээллийг авахын тулд `ps -ael` командыг ашигла. `ps` командын талаар илүү дэлгэрэнгүй мэдээлэл авахын тулд `man ps` командыг ашигла. Windows системийн *task manager* эцэг процессын ID-г үзүүлдэггүй; харин technet.microsoft.com-оос авч болох *process monitor* хэрэгсэл процессын мод үзүүлэх боломжтой.

Хариулт. Процессын мод нь процесс бүрийг өөрийн эцэг процесстой (`PPID`) холбож, анхны процесс (`PID 1`) хүртэл нь дагаж байгуулагдана (§3.3.1). Мод байгуулахад `ps -ael`-ийн гаралтаас `PID` (процессын дугаар), `PPID` (эцэг процессын дугаар), `CMD` (команд) гэсэн гурван багана хангалттай. Бусад баганаас `S` нь төлөв (`S` – унтаж байна, `R` – ажиллаж байна), `F` нь процессын тугууд юм.

Ердийн Linux дээр хэдэн зуун процесс байдаг тул мод хэт том болохоос сэргийлж жижиг боловч бодит шатлалыг ашиглав: Linux (Ubuntu 24.04) орчинд `session.sh` гэсэн bash скрипт ажиллуулж, тэр нь 2 `sleep` процесс, өөрийн `sleep` хүүхэдтэй нэг `sh -c`, өөрийн `sleep` хүүхэдтэй нэг дэд shell `( ... )`-ийг эхлүүлсний дараа `ps -ael` ажиллуулав. Доор гаралтын шаардлагатай мөрүүдийг үзүүлэв (43 цөмийн thread болон PID 1-ийн бусад хоёр системийн процессыг товчлохын тулд орхив):

```
F S   UID   PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S     0     1     0  0  80   0 -  4883 -      ?        00:00:00 process_api
4 S     0   228     1  0  80   0 -   702 do_wai ?        00:00:00 sh
0 S     0   232   228  0  80   0 -  1083 do_wai ?        00:00:00 session.sh
0 S     0   233   232  0  80   0 -   676 hrtime ?        00:00:00 sleep
0 S     0   234   232  0  80   0 -   676 hrtime ?        00:00:00 sleep
0 S     0   235   232  0  80   0 -   702 do_wai ?        00:00:00 sh
1 S     0   236   232  0  80   0 -  1083 do_wai ?        00:00:00 session.sh
0 S     0   238   236  0  80   0 -   676 hrtime ?        00:00:00 sleep
0 S     0   239   235  0  80   0 -   676 hrtime ?        00:00:00 sleep
4 R     0   240   232  0  80   0 -  1983 -      ?        00:00:00 ps
```

Мөр бүрийг `PPID`-ээр нь эцэгтэй нь холбоход (`228`→`1`, `232`→`228`, `233`, `234`, `235`, `236`, `240`→`232`, `239`→`235`, `238`→`236`) доорх мод гарна.



Зураг 4. `ps -ael`-ийн гаралтаас `PPID`-ээр байгуулсан процессын мод (Зураг 3.7-той төстэй). Тоо нь `PID`.

Тайлбар.

- `PID 1` нь энэ виртуал орчинд `process_api`. Ердийн Linux дистрибутивт энэ байр `systemd` (эсвэл `init`) байх бөгөөд Зураг 3.7-ийн үндэс нь `systemd` юм.
- `sh` (228) нь командыг ажиллуулсан shell, `session.sh` (232) нь миний скрипт. Түүний 5 хүүхэд: `sleep 300`, `sleep 301`, `sh -c` (235), дэд shell (236) болон `ps -ael` командыг өөрийг нь (240).
- `sh` (235) нь `sleep 302`-ийг (239), дэд shell (236) нь `sleep 303`-ийг (238) хүүхдээ болгосон тул гурван шатлалтай мод гарна. Дэд shell-ийн `F` тугийн утга `1` (*forked but didn't exec*) буюу `fork()` хийсэн боловч `exec()` хийгээгүй тул нэр нь эцгийнхтэй (`session.sh`) ижил байна.
- Модыг `PPID`-ээр холбох алхмыг жижиг Python скриптээр автоматжуулав. Бэлэн хэрэгслүүд: `pstree -p`, `ps -eo pid,ppid,comm --forest`; Windows дээр *Process Explorer* / *Process Monitor*.

3.10

Асуулт 3.10

UNIX ба Linux системд процессыг дуусгахтай холбоотойгоор `init` (эсвэл `systemd`) процессын гүйцэтгэх үүргийг тайлбарла.

Хариулт. Процесс дуусахад (`exit()`) түүний нөөцүүд (санах ой, нээлттэй файл, I/O буфер) чөлөөлөгдөх боловч процессын хүснэгтийн мөр (гарах төлвийн утгатай) эцэг нь `wait()` дуудтал үлддэг – ийм процесс **зомби** болно. Эцэг `wait()` дуудалгүйгээр өөрөө дуусвал хүүхдүүд нь **өнчин** (*orphan*) үлдэнэ. Үүнийг `init` (орчин үеийн Linux-т `systemd`) шийддэг (§3.3.2):

1. **Өнчин процессыг өвлөнө.** Өнчин болсон процесс бүрийн шинэ эцгээр `init` процесс (`PID 1`) томилогдоно. Иймд эцэггүй процесс байхгүй, процессын мод үргэлж бүтэн байх бөгөөд `init` нь шатлалын үндэс байна.
2. **Гарах төлвийг цуглуулна.** `init` үе үе `wait()` дуудаж, өвлөсөн өнчин процессууд дуусахад тэдний гарах төлвийг авна.
3. **Нөөцийг чөлөөлнө.** Ингэснээр өнчин (мөн зомби болсон) процессын PID болон процессын хүснэгтийн мөр чөлөөлөгдөж, зомби процессууд системд байнга үлдэхгүй.

Орчин үеийн Linux дистрибутивт `init`-ийг `systemd` сольсон бөгөөд ижил үүргийг гүйцэтгэнэ. Мөн Linux нь `systemd`-ээс өөр процессыг өнчин процесс өвлөгч (*subreaper*, `prctl(PR_SET_CHILD_SUBREAPER)`) болгож болдог.

Баталгаажуулалт. (а) Эцэг нь (`sh`) шууд дууссан `sleep` процесс (89) өнчин болж, түүний `PPID` нь 1 болсон:

```
$ ps -o pid,ppid,stat,comm -p 89
  PID  PPID STAT COMMAND
   89     1 S    sleep
$ ps -o pid,ppid,stat,comm -p 1
  PID  PPID STAT COMMAND
    1     0 SLl  process_api
```

(б) 3.18-ийн `zombie` програмаар: хүү (96) дууссан боловч эцэг (94) `wait()` дуудаагүй тул `Z` төлөвтэй байна. Эцгийг нь устгасны дараа хүү `PID 1`-д өвлөгдөж (`PPID` = 1), хэдэн секундын дотор `wait()`-ээр цэвэрлэгдэж процессын хүснэгтээс алга болсон (сүүлчийн командын гаралтад зөвхөн толгой мөр байгаа нь процесс үлдээгүйг заана):

```
$ ps -o pid,ppid,stat,comm -p 94,96
  PID  PPID STAT COMMAND
   94    85 S    zombie
   96    94 Z    zombie
$ kill -9 94
$ ps -o pid,ppid,stat,comm -p 96        # 0.3 s later
  PID  PPID STAT COMMAND
   96     1 Z    zombie
$ ps -o pid,ppid,stat,comm -p 96        # 2.8 s later
  PID  PPID STAT COMMAND
```

Энэ виртуал орчинд `PID 1`-ийн нэр `process_api` тул `init`/`systemd` биш боловч ижил үүргийг гүйцэтгэж байна.

3.11

Асуулт 3.11

Зураг 3.32-д үзүүлсэн програмаар анхны эцэг процессыг оролцуулан нийт хэдэн процесс үүсэх вэ?

```c
#include <stdio.h>
#include <unistd.h>

int main()
{
   int i;

   for (i = 0; i < 4; i++)
      fork();

   return 0;
}
```

Зураг 3.32. Хэдэн процесс үүсэх вэ?

Хариулт. Нийт **16 процесс** үүснэ (анхны эцэг процесс + 15 шинэ хүүхэд процесс).

Тайлбар: `for` давталтын 4 итераци бүрд тухайн мөчид байгаа **бүх** процесс `fork()` дуудна. Хүүхэд процесс эцгийнхээ `i` хувьсагчийн утгыг хуулбарлан авч, `fork()`-оос буцсан цэгээс үргэлжлүүлдэг тул итераци i-д үүссэн хүүхэд нь үлдсэн итерациудыг эцэгтэйгээ адил гүйцэтгэнэ. Иймд процессын тоо итераци бүрд 2 дахин өснө:

Итераци i	`fork()`-ийн өмнө	Шинээр үүссэн	`fork()`-ийн дараа
0	1	1	2
1	2	2	4
2	4	4	8
3	8	8	16

Ерөнхийдөө n итераци бүхий давталт нийт 2^n процесс үүсгэнэ: энд $2^4 = 16$, шинээр үүссэн нь $1 + 2 + 4 + 8 = 15$. (Ялгаа: 3.2-т гурван дараалсан `fork()` 8 процесс өгсөн.)

Баталгаажуулалт. Процесс бүр нэг мөр хэвлэдэг болгож (3.2-той адил, `printf("pid=%d ppid=%d\n", ...)` нэмж, эцэг нь `wait()`-ээр хүлээдэг) ажиллуулахад `./fig3_32_count | wc -l` командын гаралт **16** байв. Мөн `fork()`-ийн гүнээр (эцгээс хэдэн `fork()`-оор үүссэн) процессын тоо **1, 4, 6, 4, 1** гарсан нь биномын коэффициент $\binom{4}{k}$-тай яг таарч байна; анхны процесс 4 хүүхэдтэй (`fork()` бүрээс нэг) байв.

3.12

Асуулт 3.12

Зураг 3.33-д `printf("LINE J")` гэж тэмдэглэсэн мөрөнд ямар нөхцөлд хүрч очих вэ, тайлбарла.

```c
#include <sys/types.h>
#include <sys/wait.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

   /* fork a child process */
   pid = fork();

   if (pid < 0) { /* error occurred */
      fprintf(stderr, "Fork Failed");
      return 1;
   }
   else if (pid == 0) { /* child process */
      execlp("/bin/ls","ls",NULL);
      printf("LINE J");
   }
   else { /* parent process */
     /* parent will wait for the child to complete */
     wait(NULL);
     printf("Child Complete");
   }

   return 0;
}
```

Зураг 3.33. LINE J-д хэзээ хүрэх вэ? (<sys/wait.h>-г хөрвүүлэхийн тулд нэмсэн)

Хариулт. `execlp()` нь `exec` бүлгийн системийн дуудалт бөгөөд **дуудсан процессын хаягийн орон зайг** (код, өгөгдөл, heap, стек) **шинэ програмаар бүрэн солино** (§3.3.1). Амжилттай болбол `/bin/ls` програм эхнээс нь ажиллаж, хуучин програмын `printf("LINE J")` мөр хаягийн орон зайд үлдэхгүй тул `execlp()`-оос хэзээ ч буцахгүй. Иймд `LINE J` энэ үед **хэзээ ч хэвлэгдэхгүй**.

`printf("LINE J")` нь зөвхөн **`execlp()` амжилтгүй болж** (`-1` буцааж, `errno` тохируулсан) хүү процесс хуучин програмынхаа дараагийн мөр рүү үргэлжлэх үед ажиллана. Шалтгаан нь: `/bin/ls` файл байхгүй эсвэл зам буруу, гүйцэтгэх эрхгүй

(EACCES), файлын формат буруу (ENOEXEC), санах ой/нөөц хүрэлцэхгүй (ENOMEM) зэрэг байж болно.

Баталгаажуулалт. Хоёр тохиолдлыг харуулахын тулд програмыг зөвхөн энэ жишээнд програмын замыг командын мөрөөс (argv[1]) авдаг, гаралтад шинэ мөр нэмдэг болгож өөрчилсөн. Замыг өгөхгүй бол /bin/ls ажиллана:

```
$ ./fig3_33
a.txt
b.txt
Child Complete
$ ./fig3_33 /bin/no_such_program
LINE J
Child Complete
```

Эхний тохиолдолд ls гаралт гарч LINE J огт харагдаагүй; хоёр дахь тохиолдолд execlp() амжилтгүй болсон тул LINE J хэвлэгдсэн.

3.13

Асуулт 3.13

Зураг 3.34-ийн програмыг ашиглан A, B, C, D мөрүүд дэх `pid`-ийн утгуудыг тодорхойл. (Эцэг болон хүү процессын бодит `pid` нь тус тус 2600 ба 2603 гэж үз.)

```c
#include <sys/types.h>
#include <sys/wait.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid, pid1;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
       fprintf(stderr, "Fork Failed");
       return 1;
    }
    else if (pid == 0) { /* child process */
       pid1 = getpid();
       printf("child: pid = %d",pid); /* A */
       printf("child: pid1 = %d",pid1); /* B */
    }
    else { /* parent process */
       pid1 = getpid();
       printf("parent: pid = %d",pid); /* C */
       printf("parent: pid1 = %d",pid1); /* D */
       wait(NULL);
    }

    return 0;
}
```

Зураг 3.34. pid-ийн утгууд хэд вэ? (<sys/wait.h> нэмсэн)

Хариулт. `fork()` нь **хүү процесст 0**, **эцэг процесст шинэ хүүгийн pid-г** буцаадаг; `getpid()` нь **дуудсан процессынхаа өөрийн pid-г** буцаадаг. Иймд:

Мөр	Хэвлэгдэх мөр	Утга	Тайлбар
A	`child: pid = ...`	**0**	хүү процесс дахь `fork()`-ийн буцаасан утга
B	`child: pid1 = ...`	**2603**	хүү процессын өөрийн pid (`getpid()`)
C	`parent: pid = ...`	**2603**	эцэг процесс дахь `fork()`-ийн буцаасан утга = хүүгийн pid
D	`parent: pid1 = ...`	**2600**	эцэг процессын өөрийн pid (`getpid()`)

Баталгаажуулалт. Утга хоорондоо ялгагдахын тулд `printf` бүрт шинэ мөр нэмж ажиллуулахад (энэ удаагийн бодит pid: эцэг 135, хүү 136):

```
child: pid = 0
child: pid1 = 136
parent: pid = 136
parent: pid1 = 135
```

A = 0; B ба C нь хүүгийн pid (136) – хоёр өөр процесс дээр ижил утгатай; D нь эцгийн pid (135). Энэ нь дээрх хүснэгттэй (2600/2603) яг ижил хамааралтай.

3.14

Асуулт 3.14

Энгийн pipe нь нэрлэсэн pipe-аас илүү тохиромжтой байх нөхцөлийн нэг жишээ, мөн нэрлэсэн pipe нь энгийн pipe-аас илүү тохиромжтой байх нөхцөлийн нэг жишээг өг.

Хариулт. Энгийн (*ordinary*) pipe нь зөвхөн `fork()`-оор холбогдсон эцэг–хүүхэд процессуудын хооронд, тэднийг байх хугацаанд оршдог; нэрлэсэн (*named* pipe, UNIX-д *FIFO*) нь файлын системд нэртэй байдаг тул ямар ч процесс нэрээр нь нээж, процессууд дууссаны дараа ч үлддэг (§3.7.4).

Энгийн pipe илүү тохиромжтой нөхцөл. Эцэг–хүүхэд (эсвэл нэг эцгийн хоёр хүүхэд) хоорондын богино хугацааны, нэг чиглэлтэй өгөгдөл дамжуулалт. Жишээ нь shell дээрх `ls | wc -l`: shell pipe үүсгэж `ls`, `wc` гэсэн хоёр хүүхдийг `fork()` хийн, нэгийнх нь гаралтыг нөгөөгийнх нь оролт болгодог (мөн 3.26, 3.27-ийн програмууд). Энгийн pipe процесс дуусмагц автоматаар устдаг, файлын системд нэр үлдээдэггүй тул нэр давхцах, цэвэрлэх асуудалгүй, мөн өвлөсөн descriptor-оор л хандах боломжтой тул гаднын процесс нэвтрэхгүй – энгийн бөгөөд аюулгүй.

Нэрлэсэн pipe илүү тохиромжтой нөхцөл. Эцэг–хүүхдийн харилцаагүй, бие даасан, өөр өөр цагт эхэлсэн процессууд хооронд, эсвэл олон процесс нэг pipe ашиглах (олон бичигч, нэг уншигч), мөн процессууд дууссаны дараа ч холболт үлдэх шаардлагатай үед. Жишээ нь лог цуглуулагч демон `/var/run/app.fifo`-г уншиж, олон бие даасан програм түүнд лог бичих; эсвэл сервер нэртэй FIFO үүсгэж, ямар ч клиент түүнд хүсэлт бичих. Windows-

ийн нэрлэсэн pipe нь мөн сүлжээгээр (өөр машинтай) харилцах боломжтой.

Шинж	Энгийн pipe	Нэрлэсэн pipe (FIFO)
Эцэг–хүүхдийн харилцаа	Шаардлагатай (`fork()`-оор өвлөгдөнө)	Шаардлагагүй
Нэр	Байхгүй (файлын descriptor)	Файлын системд нэртэй (`mkfifo`)
Чиглэл	Нэг чиглэлт (хоёр чиглэлд 2 pipe)	Хоёр чиглэлт боломжтой (UNIX-д зөвхөн *half-duplex*)
Оролцогчид	Ихэвчлэн нэг бичигч, нэг уншигч	Олон процесс, олон бичигч
Оршин буй хугацаа	Процессууд дуустал	Тодорхой устгатал (`rm`) үлдэнэ

Баталгаажуулалт. (а) Хоёр бие даасан процесс FIFO-оор харилцсан нь; хоёул дууссаны дараа FIFO файл хэвээр үлдсэн (`ls -l`-ийн эхний тэмдэг `p` нь FIFO):

```
$ mkfifo /tmp/lab3_fifo
$ ls -l /tmp/lab3_fifo
prw-r--r-- 1 root root 0 Sep 21 13:40 /tmp/lab3_fifo
$ cat /tmp/lab3_fifo &        # reader
$ echo 'message from an independent writer' > /tmp/lab3_fifo   # writer
message from an independent writer
$ ls -l /tmp/lab3_fifo        # both finished, FIFO remains
prw-r--r-- 1 root root 0 Sep 21 13:40 /tmp/lab3_fifo
$ rm /tmp/lab3_fifo
```

(б) Shell-ийн энгийн pipe-ийн хоёр төгсгөл нь нэг эцгийн (shell, PID 132) хоёр хүүхэд болохыг харуулав:

```
$ sleep 5 | sleep 6 &
$ ps -o pid,ppid,comm --ppid 132
  PID  PPID COMMAND
  133   132 sleep
  134   132 sleep
  136   132 ps
```

3.15

Асуулт 3.15

RPC механизмыг авч үз. «At most once» (хамгийн ихдээ нэг удаа) болон «exactly once» (яг нэг удаа) семантикийн аль алиныг нь хэрэгжүүлээгүйгээс үүсэж болох хүсээгүй үр дагаврууды тайлбарла. Эдгээр баталгааны аль нь ч байхгүй механизмын боломжит хэрэглээг тайлбарла.

Хариулт. Локал процедурын дуудалтаас ялгаатай нь RPC нь сүлжээний ердийн алдаанаас (мессеж алдагдах, давхардах, ACK алдагдах, серверийн доголдол) болж **бүтэлгүйтэх эсвэл давхардан нэгээс олон удаа гүйцэтгэгдэж** болно (§3.8.2).

Хүсээгүй үр дагавар.

1. **«At most once» биелэхгүй бол** хүсэлт **нэгээс олон удаа** гүйцэтгэгдэж болно (клиент *timeout*-оор дахин илгээх, сүлжээнд мессеж давхардах). Давтагдсан үед ижил үр дүн өгдөггүй (идемпотент бус) үйлдэлд энэ нь хор хөнөөлтэй: банкны шилжүүлэг/төлбөр 2 удаа хийгдэх, тасалбар, захиалга давхар бүртгэгдэх, тоолуур хоёр удаа нэмэгдэх, файлын төгсгөлд өгөгдөл давхар бичигдэх – өгөгдөл зөрчигдөнө.
2. **«Exactly once» биелэхгүй бол** (сервер хүсэлтийг хүлээн авсныг ACK-аар батлах, клиент дахин илгээх байхгүй бол) хүсэлт **огт гүйцэтгэгдэхгүй** байж болно, клиент ч үр дүнг мэдэхгүй. Жишээ нь данснаас мөнгө хасагдсан боловч хүлээн авагчийн дансанд оруулах хүсэлт алдагдвал мөнгө «алга болно»; захиалга бүртгэгдээгүй боловч хэрэглэгч бүртгэгдсэн гэж боддог.
3. Хоёуланг нь хэрэгжүүлэхгүй бол клиент хүсэлтээ **0, 1 эсвэл олон удаа** гүйцэтгэгдсэн эсэхийг мэдэх аргагүй.

Ийм баталгаагүй механизмын боломжит хэрэглээ. Давхар гүйцэтгэл эсвэл алдагдал хор хөнөөлгүй, эсвэл баталгааны зардал (цагийн тэмдэг, түүх, ACK, дахин илгээлт) чухал үед:

- **Идемпотент үйлдэл** – давтагдсан ч ижил үр дүнтэй: файлын заасан офсетоос өгөгдөл унших, «идэвхтэй хэрэглэгчдийн жагсаалт» авах (§3.8.2-ын жишээ), «одоогийн цаг» асуух (§3.8.1-ийн *date server*), нэрийн хайлт (DNS).
- **Байнга давтагддаг мэдээлэл** – хэмжилтийн (sensor) утга, *heartbeat*, хяналтын статистик, тоглоомын байрлалын шинэчлэл, шууд аудио/видео: нэг мессеж алдагдвал дараагийнх нь удалгүй ирнэ, хуучин утгыг дахин илгээх нь утгагүй.
- **Апп-ын түвшинд өөрөө шалгадаг үед** – клиент үр дүнг өөрөө шалгаж дахин оролддог бол системийн баталгаа шаардлагагүй. Сервер түүх, ACK хадгалахгүй тул хурдан, энгийн, ачаалал багатай, өргөтгөхөд хялбар.

3.16

Асуулт 3.16

Зураг 3.35-д үзүүлсэн програмыг ашиглан X ба Y мөрүүд дээр ямар гаралт гарахыг тайлбарла.

```c
#include <sys/types.h>
#include <sys/wait.h>
#include <stdio.h>
#include <unistd.h>

#define SIZE 5

int nums[SIZE] = {0,1,2,3,4};

int main()
{
int i;
pid_t pid;

    pid = fork();

    if (pid == 0) {
       for (i = 0; i < SIZE; i++) {
          nums[i] *= -i;
          printf("CHILD: %d ",nums[i]); /* LINE X */
       }
    }
    else if (pid > 0) {
       wait(NULL);
       for (i = 0; i < SIZE; i++)
          printf("PARENT: %d ",nums[i]); /* LINE Y */
    }

    return 0;
}
```

Зураг 3.35. X ба Y мөрөнд ямар гаралт гарах вэ? (<sys/wait.h> нэмсэн)

Хариулт. `fork()`-ийн дараа хүүхэд процесс глобал массив `nums`-ийн **өөрийн хуулбарыг** авна (3.1-тэй адил). Иймд:

- **LINE X** (хүүхэд): `nums[i] *= -i` нь $0 \cdot 0,\ 1 \cdot (-1),\ 2 \cdot (-2),\ 3 \cdot (-3),\ 4 \cdot (-4)$ гэж тооцоолж, өөрийн хуулбарыг `0, -1, -4, -9, -16` болгоно. Хэвлэх нь: `CHILD: 0 CHILD: -1 CHILD: -4 CHILD: -9 CHILD: -16`

- **LINE Y** (эцэг): `wait(NULL)`-аар хүүхдийг дуустал хүлээсэн ч эцгийн `nums` хуулбар өөрчлөгдөөгүй тул анхны утгаа хэвээр хэвлэнэ. Хэвлэх нь: `PARENT: 0 PARENT: 1 PARENT: 2 PARENT: 3 PARENT: 4`

`printf`-д шинэ мөр (`\n`) байхгүй тул бүх гаралт нэг мөрөнд нийлж гарна; эхлээд хүүхдийнх, дараа нь `wait()`-ийн улмаас эцгийнх гарна.

Баталгаажуулалт. Програмыг өөрчлөлтгүй ажиллуулахад:

```
CHILD: 0 CHILD: -1 CHILD: -4 CHILD: -9 CHILD: -16 PARENT: 0 PARENT: 1
    PARENT: 2 PARENT: 3 PARENT: 4
```

Хэрэв эдгээр нь процесс биш, нэг хаягийн орон зайг хуваалцдаг thread байсан бол массив нэг л хуулбартай тул эцэг ч `0, -1, -4, -9, -16`-г хэвлэх байсан.

3.17

Асуулт 3.17

Дараах сонголт бүрийн давуу болон сул талыг тайлбарла. Системийн түвшин болон програмистын түвшин хоёуланг нь авч үз.

a. Синхрон ба асинхрон харилцаа
b. Автомат ба тодорхой (*explicit*) буферлэлт
c. Хуулбараар илгээх ба лавлагаагаар (*reference*) илгээх
d. Тогтмол хэмжээтэй ба хувьсах хэмжээтэй мессеж

Хариулт. Дор **Систем** нь цөмд гэсэн хэрэгжүүлэлтийн, **Програмист** нь аппликейшн бичигчийн талын нөлөөг тус тус заана; + нь давуу, − нь сул тал.

a. Синхрон ба асинхрон харилцаа (блоклодог ба блоклодоггүй `send()` / `receive()`, §3.6.2)

Синхрон (блоклодог)

+ **Програмист:** логик энгийн – `send()` буцаж ирсэн бол мессеж хүлээн авагдсан гэж үзнэ; илгээгч ба хүлээн авагч уулзаж (*rendezvous*) дамжуулдаг тул нэмэлт синхрончлол хэрэггүй; producer–consumer бодлого маш энгийн болно.
+ **Систем:** мессежийг түр хадгалах буфер шаардлагагүй (нөөц хэмнэнэ), хэрэгжүүлэлт энгийн.
− **Систем ба програмист:** илгээгч эсвэл хүлээн авагч хүлээлтэнд орж зэрэгцээ ажиллагаа, дамжуулах чадвар буурна; удаан хүлээн авагч илгээгчийг саатуулна; хоёр процесс бие биедээ зэрэг блоклодог `send()` хийвэл мухардалд (*deadlock*) орно. Уулзалт шаардлагагүй үед блоклолт нь илүү зардал.

Асинхрон (блоклодоггүй)

+ **Програмист ба систем:** илгээгч шууд үргэлжлэн ажиллана – тооцоолол ба харилцаа зэрэгцэж, CPU-ийн ашиглалт, хариу өгөх чадвар өндөр (сервер, UI).

– **Систем:** мессежийг түр хадгалах дараалал (буфер) хэрэгтэй, түүнийг удирдах шаардлагатай; дараалал дүүрэх асуудалтай.

– **Програмист:** мессеж хүрсэн эсэхийг *polling*, *callback* эсвэл тасалдлаар шалгах ёстой; блоклодоггүй `receive()` «мессеж алга» (null) буцааж болох тул давтан шалгах логик хэрэгтэй; алдаа хожуу мэдэгдэх тул програмчлал, дибаг хэцүү.

Аль алиных нь давуу тал байдаг тул системүүд ихэвчлэн хоёуланг нь дэмждэг.

b. Автомат ба тодорхой буферлэлт (§3.6.3)

Автомат буферлэлт – систем мессежийн дарааллыг өөрөө удирдана (хязгаартай эсвэл хязгааргүй багтаамжтай)

+ **Програмист:** буфер хуваарилах, хэмжээг тодорхойлох шаардлагагүй – програм энгийн, алдаа цөөн; хязгааргүй дарааллын үед илгээгч хэзээ ч блоклогдохгүй.

+ **Систем:** илгээгч ба хүлээн авагчийг бие биенээс нь салгаж асинхрон ажиллуулах боломж олгоно.

– **Систем:** санах ой үрэгдэж болно; хязгааргүй дараалал хүлээн авагч удаан үед цөмийн санах ойг барж болно; дараалал удирдах нэмэлт ажил. Хязгаартай дараалал дүүрвэл илгээгч блоклогдоно.

– **Програмист:** буферийн хэмжээ, зан төлвийг хянах боломж бага, гүйцэтгэлийг урьдчилан таамаглахад хэцүү; мессеж нэмэлт хуулагдана (илгээгч → системийн буфер → хүлээн авагч).

Тодорхой буферлэлт – буферийн хэмжээг програмист заана

+ **Програмист:** ачаалалдаа тааруулж хэмжээг тохируулах, санах ойг хэмнэх боломжтой; санах ойн хэрэглээ урьдчилан тодорхой.

+ **Систем:** нөөцийн хэрэглээг хязгаарлана (хязгааргүй санах ой нөөцлөх шаардлагагүй).

– **Програмист:** буферийг өөрөө хуваарилж, чөлөөлж, хэмжээг сонгох ёстой – буфер халих, санах ойн алдагдал гарах эрсдэлтэй; хэт жижиг бол дүүрч илгээгч блоклогдоно, хэт том бол санах ой үрэгдэнэ.

– **Систем:** буфер дүүрвэл илгээгчийг блоклох эсвэл мессежийг хаях бодлого шаардлагатай.

c. Хуулбараар илгээх ба лавлагаагаар илгээх

Хуулбараар илгээх (send by copy)

+ **Програмист:** хүлээн авагч өөрийн хуулбарыг авдаг тул илгээгчийн өгөгдлийг өөрчилж чадахгүй; илгээгч `send()`-ийн дараа буферээ шууд дахин ашиглаж болно; өрсөлдөөний нөхцөл (*race condition*) үүсэхгүй.

+ **Систем:** процессууд ялгаатай хаягийн орон зайд, тэр байтугай өөр машинд байвал (сүлжээ) заавал хуулбар хийнэ – тархсан системд тохиромжтой; тусгаарлалт, хамгаалалт сайн.

– **Систем ба програмист:** хуулах цаг, санах ойн зардал өгөгдлийн хэмжээтэй пропорционал өснө – том өгөгдөлд удаан (илгээгч, системийн буфер, хүлээн авагч гэсэн хэд хэдэн хуулбар).

Лавлагаагаар илгээх (send by reference)

+ **Систем:** хуулалт хийхгүй – хурдтай, санах ой хэмнэнэ, мессежийн хэмжээнээс үл хамаарч тогтмол хугацаа.
+ **Програмист:** хүлээн авагч илгээгчийн объектыг шууд өөрчилж чадах тул төвлөрсөн програмыг тархсан хувилбар болгоход хялбар (жишээ нь Java RMI-ийн алсын объект).
– **Програмист:** хоёр тал ижил өгөгдлийг хуваалцдаг тул синхрончлол (өрсөлдөөний нөхцөл) шаардлагатай; хүлээн авагч илгээгчийн өгөгдлийг эвдэж болно; өгөгдлийн эзэмшил, амьдрах хугацаа (хэн чөлөөлөх, «унжсан» заагч) төвөгтэй.
– **Систем:** өөр хаягийн орон зай, өөр машинд заагч утгагүй тул хуваалцсан санах ой эсвэл тархсан объектын дэмжлэг хэрэгтэй; хамгаалалт сул.

d. Тогтмол хэмжээтэй ба хувьсах хэмжээтэй мессеж (§3.6)

Тогтмол хэмжээтэй

+ **Систем:** хэрэгжүүлэлт энгийн – дарааллын нүд бүр ижил хэмжээтэй тул хуваарилалт хурдан, санах ой хэлтэрхийжихгүй, буферт хэдэн мессеж багтах нь урьдчилан тодорхой (нөөцийн хэрэглээг таамаглаж болно).
– **Програмист:** том өгөгдлийг олон мессежид хувааж, хүлээн авагч дээр дахин угсрах ёстой – програмчлал хэцүү; жижиг мессежид хэмжээ дүүргэх зай үрэгдэнэ.

Хувьсах хэмжээтэй

+ **Програмист:** ямар ч хэмжээтэй өгөгдлийг нэг мессежээр илгээж болно – програмчлал хялбар; зай үрэхгүй.
– **Систем:** хэрэгжүүлэлт нарийн – динамик хуваарилалт, санах ой хэлтэрхийжих, мессежийн уртыг тэмдэглэж шалгах шаардлагатай; буферт хэдэн мессеж багтахыг урьдчилан хэлэх боломжгүй.

Жишээ: Windows-ийн ALPC жижиг мессежийг (256 байт хүртэл) хуулж дамжуулдаг бол том мессежийг хуваалцсан санах ойгоор (*section object*) дамжуулдаг (§3.7.3).

Дүгнэлт. Дөрвөн сонголт бүрд системд энгийн нь програмистад төвөгтэй, програмистад хялбар нь системд нарийн байдаг. Ном үүнийг үйлдлийн системийн зохиомжид байнга тохиолддог **солилцоо** (*tradeoff*) гэж нэрлэдэг (§3.6).

3 Дүгнэлт

3-р бүлгийн Practice Exercises 3.1–3.17-г гүйцэтгэх явцад дараах гол дүгнэлтэд хүрэв:

- `fork()` нь эцэг процессын хаягийн орон зайн хуулбарыг үүсгэдэг тул эцэг, хүүхдийн өгөгдөл (глобал хувьсагч, стек, heap) хоорондоо хамааралгүй; зөвхөн хуваалцсан санах ойн сегмент хуваалцагдана (3.1, 3.5).
- Дараалсан n ширхэг `fork()` нийт 2^n процесс үүсгэнэ (3.2).
- Зэрэгцээ боловсруулалт нь хуваарилалт, санах ойн хамгаалалт, синхрончлолын хүндрэл дагуулдаг (3.3). Контекст сэлгэлтийн хурд техник хангамжийн дэмжлэгээс (олон регистрийн багц) хамаарна (3.4).
- RPC-ийн «яг нэг удаа» семантик нь цагийн тэмдэг ба түүх, ACK ба дахин илгээлт дээр тулгуурладаг; серверийн доголдлыг тэсвэрлэхийн тулд тогтвортой хадгалалт дахь лог шаардлагатай (3.6, 3.7).
- Контекст сэлгэлт нь PCB-д төлөв хадгалж, шинийг сэргээх цэвэр нэмэгдэл зардал (3.8). Процессын мод `PPID`-ээр холбогддог (3.9); өнчин процессыг `init` /`systemd` өвлөж, `wait()`-ээр цэвэрлэдэг (3.10).
- `for` давталт дахь `fork()` процессын тоог итераци бүрд 2 дахин өсгөнө (3.11); `exec()` амжилттай бол буцахгүй, зөвхөн алдаатай үед л дараагийн мөр ажиллана (3.12); `fork()` хүүд 0, эцэгт хүүгийн pid-г буцаана (3.13); процессууд өгөгдлөө тус тусдаа хуулбарлан авдаг (3.16).
- Энгийн pipe нь эцэг–хүүхдийн богино хугацааны, нэрлэсэн pipe нь бие даасан процессуудын урт хугацааны харилцаанд тохиромжтой (3.14). RPC-д «at most once» / «exactly once» баталгаагүй бол давхар эсвэл алдагдсан гүйцэтгэл гарна; идемпотент үйлдэлд баталгаагүй механизм хангалттай (3.15).
- Мессеж дамжуулалтын сонголт бүр (синхрон/асинхрон, автомат/тодорхой буфер, хуулбар/лавлагаа, тогтмол/хувьсах хэмжээ) системийн энгийн байдал ба програмчлалын хялбар байдлын хоорондох солилцоо юм (3.17).

Ашигласан материал

[1] A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th Edition. John Wiley & Sons, 2018. Chapter 3: Processes (§3.2.3 Context Switch, §3.3 Operations on Processes, §3.6 IPC in Message-Passing Systems, §3.7.1 POSIX Shared Memory, §3.7.3 Windows, §3.7.4 Pipes, §3.8.2 Remote Procedure Calls; Practice Exercises 3.1–3.7, x. 154–155; Exercises 3.8–3.17, x. EX-4–EX-7).

[2] Linux manual pages: `ps(1)`, `pstree(1)`, `mkfifo(1)`, `fork(2)`, `wait(2)`, `execlp(3)`, `getpid(2)`.